1. PyQT的多线程

进程：

- 是指一个内存中运行的应用程序，每个进程都有一个独立的内存空间，一个应用程序可以同时运行多个进程；
- 进程也是程序的一次执行过程，是系统运行程序的基本单位；系统运行一个程序即是 一个进程从创建、运行到消亡的过程。

线程：

- 线程是进程中的一个执行单元，负责当前进程中程序的执行，
- 一个进程中**至少**有一个线程。
- 一个进程中是可以有多个线程的，这个应用程序也可以称之为多线程程序。

在PyQT中如何创建线程：

```
1. 创建一个自定义线程类，继承于QThread
2. 重写run函数---线程业务处理函数
3. 创建自定义线程类的对象
4. 启动线程
5. 线程退出
6. 线程销毁
```

案例：

```
打开摄像头，实时显示彩色画面与灰度画面，并且录制视频
```

思路：

```
启动两个子线程，界面运行起来就是一个主线程
子线程1实时读取摄像头画面信息，将信息进行处理后发回主线程，主线程需要在标签上进行显示
    主线程如何获取到子线程中的画面数据：
        子线程1通过发信号的方式把图像数据给主线程，主线程中的槽函数接收画面数据，在标签上显示
子线程2获取子线程1的信号图像数据，使用 cv2.VideoWriter()保存视频
```

注意：

OpenCV这个三方库，在后期的视频中会进行详细教学，这里只是简单涉及，在这里关于这两个函数，大家仅了解使用即可。

```
在此打开摄像头使用的是 python中的 OpenCV实现的。
OpenCV获取到的图像数据是 Mat矩阵,本质上是 Numpy 数组，图像中的像素通道顺序是BGR
但在界面中通过标签显示的图像数据是 QPixmap类型，图像中的像素通道顺序是RGB，所以我们需要借助
QImage过渡一下
    需要把 Mat(BGR)->QPixmap(RGB);但是没有直接转换的方式
    需要把 Mat(BGR)->QImage(RGB)->QPixmap(RGB)
在此需要使用到Opencv中的图像处理函数：
    cv2.flip()       镜像处理函数
    cv2.cvtColor()   像素转换函数
```

华清远见元宇宙实验中心 yvlab.com.cn

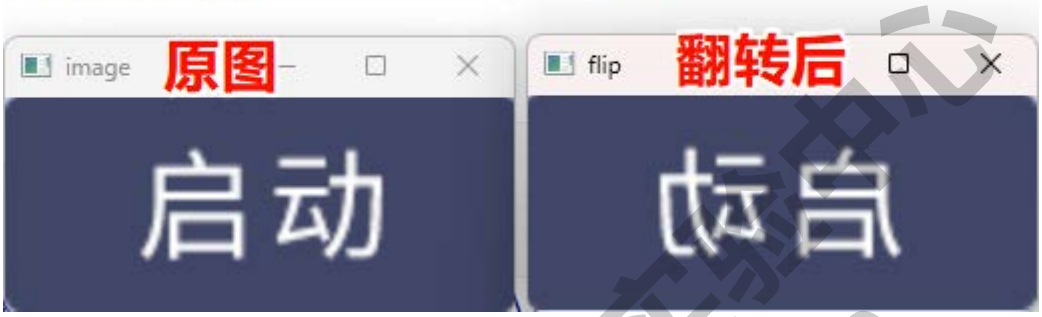
```python
import cv2

image = cv2.imread("../images/btn.png")
cv2.imshow( winname: "image", image)

img = cv2.flip(image,  flipCode: 1)
cv2.imshow( winname: "flip", img)

cv2.waitKey(0)
```

镜像规则：
0：以X轴翻转
1：以Y轴翻转
-1：同时以X轴和Y轴翻转



cvtColor() 像素转换函数

```python
import cv2

image = cv2.imread("../images/btn.png")
cv2.imshow( winname: "bgr", image)

img = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
cv2.imshow( winname: "grap", img)

img = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
cv2.imshow( winname: "rgb", img)

cv2.waitKey(0)
```

2. 总结

2.1 QThread

在 PyQt5 中，`QThread` 是一个用于实现多线程的类。它属于 `PyQt5.QtCore` 模块，主要用于在 GUI 应用程序中执行耗时任务而不阻塞主线程（即 GUI 线程）。通过使用 `QThread`，可以确保应用程序的界面保持响应性，同时在后台执行复杂的计算、网络请求或其他耗时操作。

`QThread` 的主要作用是为 PyQt5 应用程序提供多线程支持，通常用于以下场景：

- **后台任务**：执行耗时操作（如文件处理、网络请求）而不阻塞主线程。
- **保持界面响应性**：避免因长时间运行的任务导致 GUI 冻结。

华清远见元宇宙实验中心 yyzlab.com.cn

- **异步编程**：在独立线程中运行任务，并通过信号与槽机制与主线程通信。
- **资源管理**：在线程中管理资源（如定时器、文件句柄等），并在任务完成后释放资源。

2.1.1 `QThread` 的导入方式

```
from PyQt5.QtCore import QThread
```

2.1.2 常用方法

2.1.2.1 创建线程

- `QThread(parent=None)`：创建一个新的线程对象。

  ```
  thread = QThread()
  ```

2.1.2.2 启动线程

- `start()`：启动线程并调用其 `run()` 方法。

  ```
  thread.start()
  ```

2.1.2.3 停止线程

- `quit()`：请求线程退出。

  ```
  thread.quit()
  ```

- `terminate()`：强制终止线程（不推荐使用，可能导致资源泄漏）。

  ```
  thread.terminate()
  ```

2.1.2.4 检查线程状态

- `isRunning()`：检查线程是否正在运行。

```
if thread.isRunning():
    print("线程正在运行")
```

- `isFinished()`：检查线程是否已完成。

  ```
  if thread.isFinished():
      print("线程已完成")
  ```

2.1.2.5 等待线程完成

- `wait(unsigned long time=ULONG_MAX)`：阻塞当前线程，直到目标线程完成或超时。

  ```
  thread.wait()  # 等待线程完成
  ```

华清远见|元宇宙实验中心
yyzlab.com.cn

2.1.2.6 自定义线程逻辑

- **重写 `run()` 方法**：在子类中重写 `run()` 方法以定义线程的具体任务。

```
class WorkerThread(QThread):
    def run(self):
        for i in range(5):
            print(f"工作线程: {i}")
```

2.1.2.7 信号与槽

- `finished.connect(slot)`：当线程完成时触发。

```
thread.finished.connect(lambda: print("线程已完成"))
```

- `started.connect(slot)`：当线程启动时触发。

```
thread.started.connect(lambda: print("线程已启动"))
```

2.1.3 典型应用场景

- **后台任务**：执行耗时操作（如文件下载、图像处理）而不阻塞主线程。
- **异步通信**：通过信号与槽机制在不同线程之间传递数据。
- **定时任务**：在线程中运行定时器或循环任务。
- **资源管理**：在线程中管理资源，并在任务完成后释放资源。

2.2 QImage

在 PyQt5 中，`QImage` 是一个用于处理图像数据的类。它属于 `PyQt5.QtGui` 模块，主要用于加载、保存和操作图像文件（如 PNG、JPEG 等）。`QImage` 提供了丰富的功能，例如调整图像大小、转换格式、像素操作等，并支持与 `QPixmap` 和其他 GUI 控件结合使用。

`QImage` 的主要作用是为应用程序提供图像处理功能，通常用于以下场景：

- **加载和保存图像**：从文件或内存中加载图像，并将其保存到文件。
- **图像操作**：调整图像大小、裁剪、旋转、缩放等。
- **像素级操作**：直接访问和修改图像的像素数据。
- **格式转换**：将图像转换为不同的格式（如 RGB、灰度图等）。
- **与其他控件结合**：与 `QLabel` 或 `QGraphicsView` 结合显示图像。

2.2.1 `QImage` 的导入方式

```
from PyQt5.QtGui import QImage
```

2.2.2 常用方法

2.2.2.1 创建图像对象

- `QImage()`：创建一个空的图像对象。

```
image = QImage()
```

- `QImage(str fileName)`：从文件加载图像。

华清远见学习实验中心 yyzlab.com.cn

```
image = QImage("example.png")
```

- `QImage(data, width, height, format)`：从原始数据创建图像。
  - 参数：
    - `data`：图像的字节数据。
    - `width`：图像的宽度。
    - `height`：图像的高度。
    - `format`：图像的格式（如 `QImage.Format_RGB32`）。

```
from PyQt5.QtGui import QImage

data = b"\xFF\x00\x00" * 100  # 示例数据
image = QImage(data, 10, 10, QImage.Format_RGB32)
```

2.2.2.2 加载和保存图像

- `load(str fileName)`：从文件加载图像。

```
image.load("example.jpg")
```

- `save(str fileName, format=None, quality=-1)`：将图像保存到文件。
  - 参数：
    - `fileName`：保存的文件名。
    - `format`：保存的格式（可选，默认根据文件扩展名推断）。
    - `quality`：保存的质量（仅适用于 JPEG 格式）。

```
image.save("output.png")
```

2.2.2.3 获取图像信息

- `size()`：获取图像的尺寸（返回 `QSize` 对象）。

```
size = image.size()
print(f"宽度: {size.width()}, 高度: {size.height()}")
```

- `width()` 和 `height()`：分别获取图像的宽度和高度。

```
print(f"宽度: {image.width()}, 高度: {image.height()}")
```

- `format()`：获取图像的格式。

```
print(image.format())
```

2.2.2.4 调整图像大小

- `scaled(width, height, aspectRatioMode=Qt.IgnoreAspectRatio, transformMode=Qt.FastTransformation)`：调整图像大小。
  - 参数：
    - `aspectRatioMode`：宽高比模式（如 `Qt.KeepAspectRatio`）。

华清远见元宇宙实验中心 yyzlab.com.cn

- `transformMode`：变换模式（如 `Qt.SmoothTransformation`）。

```
from PyQt5.QtCore import Qt

scaled_image = image.scaled(200, 200, Qt.KeepAspectRatio,
Qt.SmoothTransformation)
```

2.2.2.5 裁剪图像

- `copy(QRect rect)`：裁剪图像的指定区域。

```
cropped_image = image.copy(50, 50, 100, 100)  # 裁剪 (50, 50) 到 (150, 150) 的
区域
```

2.2.2.6 像素操作

- `pixel(x, y)`：获取指定位置的像素颜色。

```
color = image.pixel(10, 10)
print(color)
```

- `setPixel(x, y, QColor color)`：设置指定位置的像素颜色。

```
from PyQt5.QtGui import QColor

image.setPixel(10, 10, QColor(255, 0, 0).rgb())  # 设置为红色
```

2.2.2.7 图像格式转换

- `convertToFormat(QImage.Format format)`：将图像转换为指定格式。

```
gray_image = image.convertToFormat(QImage.Format_Grayscale8)
```

2.2.2.8 检查图像有效性

- `isNull()`：检查图像是否为空。

```
if image.isNull():
    print("图像无效")
```

2.2.3 典型应用场景

- **图像加载和保存**：从文件加载图像并保存到不同格式。
- **图像编辑**：调整大小、裁剪、旋转、缩放等操作。
- **像素级处理**：对图像的每个像素进行操作（如滤镜效果）。
- **格式转换**：将图像转换为不同的格式（如 RGB、灰度图等）。
- **显示图像**：与 `QLabel` 或 `QGraphicsView` 结合显示图像。

2.3 QFrame

华清远见元宇宙实验中心 yyzlab.com.cn

在 PyQt5 中，`QFrame` 是一个用于创建框架控件的类。它属于 `PyQt5.QtWidgets` 模块，主要用于为其他控件提供装饰性边框或分隔线。`QFrame` 提供了多种样式和形状选项，可以用来增强 GUI 界面的视觉效果。

`QFrame` 的主要作用是为应用程序提供装饰性框架或分隔线，通常用于以下场景：

- **边框装饰**：为控件添加边框以突出显示。
- **分隔线**：在界面中添加水平或垂直分隔线。
- **自定义形状**：支持矩形、线条、面板等多种形状。
- **阴影效果**：通过设置阴影样式增强视觉效果。
- **嵌套布局**：与其他布局结合使用，创建复杂的界面结构。

2.3.1 `QFrame` 的导入方式

```
from PyQt5.QtWidgets import QFrame
```

2.3.2 常用方法

2.3.2.1 创建框架

- `QFrame(parent=None)`：创建一个新的框架控件。

  ```
  frame = QFrame()
  ```

2.3.2.2 设置框架样式

- `setFrameStyle(int style)`：设置框架的样式。
  - 参数：
    - `style`：由 `frameShape` 和 `frameShadow` 组成的组合值。
      - `frameShape`：框架形状（如 `QFrame.Box`、`QFrame.Panel`、`QFrame.HLine` 等）。
      - `frameShadow`：框架阴影（如 `QFrame.Plain`、`QFrame.Raised`、`QFrame.Sunken` 等）。

```
from PyQt5.QtWidgets import QFrame

frame.setFrameStyle(QFrame.Box | QFrame.Raised)  # 带有凸起效果的矩形框
```

2.3.2.3 设置框架形状

- `setFrameShape(QFrame.Shape shape)`：设置框架的形状。
  - 常用值：
    - `QFrame.NoFrame`：无框架。
    - `QFrame.Box`：矩形框。
    - `QFrame.Panel`：面板样式。
    - `QFrame.HLine`：水平分隔线。
    - `QFrame.VLine`：垂直分隔线。

```
frame.setFrameShape(QFrame.HLine)  # 设置为水平分隔线
```

华清远见元宇宙实验中心 yyzlab.com.cn

2.3.2.4 设置框架阴影

- `setFrameShadow(QFrame.Shadow shadow)`：设置框架的阴影样式。
  - 常用值：
    - `QFrame.Plain`：无阴影。
    - `QFrame.Raised`：凸起效果。
    - `QFrame.Sunken`：凹陷效果。

```
frame.setFrameShadow(QFrame.Sunken)  # 设置为凹陷效果
```

2.3.2.5 设置线宽

- `setLineWidth(int width)`：设置框架的线宽（仅适用于某些样式）。

```
frame.setLineWidth(2)  # 设置线宽为 2 像素
```

- `setMidLineWidth(int width)`：设置中间线的宽度（仅适用于某些样式）。

```
frame.setMidLineWidth(1)  # 设置中间线宽为 1 像素
```

2.3.2.6 嵌套布局

- **将 `QFrame` 添加到布局中**：`QFrame` 可以作为容器，包含其他控件。

```
from PyQt5.QtWidgets import QVBoxLayout, QPushButton

layout = QVBoxLayout()
layout.addWidget(QPushButton("按钮 1"))
layout.addWidget(QPushButton("按钮 2"))

frame.setLayout(layout)
```

2.3.3 典型应用场景

- **边框装饰**：为控件添加边框以突出显示。
- **分隔线**：在界面中添加水平或垂直分隔线。
- **自定义形状**：支持矩形、线条、面板等多种形状。
- **阴影效果**：通过设置阴影样式增强视觉效果。
- **嵌套布局**：与其他布局结合使用，创建复杂的界面结构。

仅清远见元宇宙实验中心 www.yzlab.com.cn